

Introduction to Python Programming

Chapter 16: Classes - Intermediate

Introduction

Now that you understand classes as templates and how to create objects, it's time to make those objects **do things**. In this chapter, you'll learn how to add methods (functions) to classes, understand `self` deeply, and use special methods that give your objects powerful behaviors.

1. Adding Methods to Classes

In Chapter 12, we learned that classes store data. Now we'll make classes perform actions.

Methods are functions that belong to a class. They can access and modify the object's data.

Basic Method Example

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} says: Woof! Woof!")

    def describe(self):
        print(f"{self.name} is a {self.breed}")

# Create a dog
my_dog = Dog("Buddy", "Golden Retriever")

# Call methods
my_dog.bark()      # Buddy says: Woof! Woof!
my_dog.describe() # Buddy is a Golden Retriever
```

Key observations:

- Methods are defined inside the class
- Methods always have `self` as the first parameter
- Methods can access instance variables through `self`
- Call methods using dot notation: `object.method()`

Methods Can Use Instance Data

```
class BankAccount:
    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount
        print(f"Deposited ${amount}. New balance: ${self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds!")
        else:
            self.balance = self.balance - amount
            print(f"Withdrew ${amount}. New balance: ${self.balance}")

# Use the account
account = BankAccount("123456", 1000)
account.deposit(500)    # Deposited $500. New balance: $1500
account.withdraw(200)  # Withdrew $200. New balance: $1300
```

Methods have access to the object's **state** - all the data stored in instance variables.

2. Understanding `self` - Two Approaches

`self` is crucial but can be confusing. Here are two different ways to understand it:

Approach 1: `self` Means "This Specific Object"

When you write a method, you're writing instructions that could apply to ANY object of that class. `self` is Python's way of saying "use THIS specific object's data."

Think of it like a form letter:

```
Dear [SELF],
Your account balance is [SELF's balance].
```

When you send this to specific people:

- For Alice: "Dear Alice, Your account balance is \$1000"
- For Bob: "Dear Bob, Your account balance is \$2000"

In code:

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"Hi, I'm {self.name} and I'm {self.age} years old")

person1 = Person("Alice", 25)
person2 = Person("Bob", 30)

person1.introduce() # self = person1, so uses person1's name and age
person2.introduce() # self = person2, so uses person2's name and age

```

When `person1.introduce()` runs:

- `self` refers to `person1`
- `self.name` is "Alice"
- `self.age` is 25

Approach 2: Python Passes the Object Automatically

Here's what actually happens behind the scenes:

When you write:

```
my_dog.bark()
```

Python internally does this:

```
Dog.bark(my_dog) # Passes my_dog as first argument
```

The object itself is passed as the first argument!

That's why every method needs `self` as the first parameter - Python automatically passes the object there.

```

class Counter:
    def __init__(self, value):
        self.value = value

    def increment(self):
        self.value = self.value + 1

counter = Counter(0)
counter.increment() # Python actually calls: Counter.increment(counter)

```

When you call `counter.increment()`:

1. Python looks up the `increment` method in the `Counter` class
2. Automatically passes `counter` as the first argument
3. Inside the method, `self = counter`
4. So `self.value` accesses `counter.value`

Key insight: You write `self` to receive the object that Python automatically sends.

Why It's Called `self`

Note: In other programming languages (JavaScript, Java, C++), this same concept uses the keyword `this` instead of `self`. Python uses `self` by convention (it's not a keyword - you could technically name it anything, but **always** use `self` for clarity).

```
// JavaScript equivalent
class Dog {
  constructor(name) {
    this.name = name; // 'this' instead of 'self'
  }

  bark() {
    console.log(this.name + " barks!"); // 'this' instead of 'self'
  }
}
```

The intent is identical across languages - referring to the current object.

3. State: The Object's Data

The **state** of an object is all its current data (instance variables) at any given moment.

```
class TrafficLight:
    def __init__(self):
        self.color = "red" # Initial state

    def change(self):
        if self.color == "red":
            self.color = "green"
        elif self.color == "green":
            self.color = "yellow"
        else:
            self.color = "red"

    def show(self):
```

```

        print(f"Light is: {self.color}")

light = TrafficLight()
light.show()    # Light is: red (initial state)
light.change()
light.show()    # Light is: green (state changed)
light.change()
light.show()    # Light is: yellow (state changed again)

```

State = the values of all instance variables at any moment.

Methods can:

- **Read** the state (`self.color`)
- **Change** the state (`self.color = "green"`)
- **Use** the state to make decisions

4. Methods That Return Values

Methods can return values just like regular functions:

```

class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def is_square(self):
        return self.width == self.height

rect = Rectangle(5, 10)
print(f"Area: {rect.area()}")           # Area: 50
print(f"Perimeter: {rect.perimeter()}") # Perimeter: 30
print(f"Is square: {rect.is_square()}") # Is square: False

```

5. Special Methods: `__str__` and `__repr__`

Python has special methods (with double underscores) that customize object behavior. Let's learn two important ones:

The Problem: Ugly Default Output

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

my_dog = Dog("Buddy", "Golden Retriever")
print(my_dog)  # <__main__.Dog object at 0x7f8b3c5d0a90>
```

That output isn't helpful! Let's fix it.

`__str__`: For Human-Readable Output

The `__str__` method defines what `print()` shows:

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def __str__(self):
        return f"{self.name} ({self.breed})"

my_dog = Dog("Buddy", "Golden Retriever")
print(my_dog)  # Buddy (Golden Retriever)
```

When is `__str__` called?

- When you use `print(object)`
- When you use `str(object)`
- Whenever Python needs a "nice" string representation

`__repr__`: For Developer-Friendly Output

The `__repr__` method provides a detailed, unambiguous representation - often showing how to recreate the object:

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def __repr__(self):
        return f'Dog(\'{self.name}\', \' {self.breed}\')
```

```
def __str__(self):
    return f"{self.name} ({self.breed})"

my_dog = Dog("Buddy", "Golden Retriever")
print(my_dog)          # Buddy (Golden Retriever) - uses __str__
print(repr(my_dog))    # Dog('Buddy', 'Golden Retriever') - uses __repr__
```

When is `__repr__` called?

- When you use `repr(object)`
- In the interactive interpreter when you type the object name
- When `__str__` isn't defined (falls back to `__repr__`)

Difference Between `__str__` and `__repr__`

- `__str__`: For end users - readable, friendly
- `__repr__`: For developers - precise, unambiguous, shows how to recreate

```
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year

    def __str__(self):
        return f'"{self.title}" by {self.author}'

    def __repr__(self):
        return f"Book('{self.title}', '{self.author}', {self.year})"

book = Book("Python Programming", "John Smith", 2024)
print(book)          # "Python Programming" by John Smith
print(repr(book))    # Book('Python Programming', 'John Smith', 2024)
```

6. Other Special Methods (Brief Preview)

Python has many special methods (also called "dunder methods" - double underscore). Here are a few you'll learn later:

- `__len__`: Define what `len(object)` returns
- `__eq__`: Define how objects compare with `==`
- `__lt__`, `__gt__`: Define `<` and `>` comparisons

- `__add__`: Define what `+` does with objects
- `__getitem__`: Make objects indexable with `[]`

Example preview:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2 # Uses __add__
print(v3)    # (4, 6)
```

We'll cover these in detail in advanced classes (Chapter 14).

7. Practical Examples

Example 1: Complete Student Class

```
class Student:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)
        print(f"Added grade {grade} for {self.name}")

    def average(self):
        if len(self.grades) == 0:
            return 0
        return sum(self.grades) / len(self.grades)

    def __str__(self):
        return f"{self.name} (ID: {self.student_id})"

    def __repr__(self):
```



```

        return f"Student('{self.name}', '{self.student_id}')"

# Use the class
student = Student("Alice", "001")
student.add_grade(85)
student.add_grade(90)
student.add_grade(92)

print(student)                # Alice (ID: 001)
print(f"Average: {student.average():.2f}") # Average: 89.00
print(repr(student))          # Student('Alice', '001')

```

Example 2: Shopping Cart

```

class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_item(self, name, price, quantity):
        self.items.append({
            "name": name,
            "price": price,
            "quantity": quantity
        })
        print(f"Added {quantity}x {name}")

    def total(self):
        total = 0
        for item in self.items:
            total += item["price"] * item["quantity"]
        return total

    def __str__(self):
        if len(self.items) == 0:
            return "Cart is empty"

        result = "Shopping Cart:\n"
        for item in self.items:
            result += f"  {item['name']}: ${item['price']} x {item['quantity']}\n"
        result += f"Total: ${self.total():.2f}"
        return result

cart = ShoppingCart()
cart.add_item("Apple", 1.99, 5)
cart.add_item("Banana", 0.99, 3)
print(cart)

```

Example 3: Temperature Converter

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    def to_fahrenheit(self):
        return (self.celsius * 9/5) + 32

    def to_kelvin(self):
        return self.celsius + 273.15

    def __str__(self):
        return f"{self.celsius}°C"

    def __repr__(self):
        return f"Temperature({self.celsius})"

temp = Temperature(25)
print(temp)                # 25°C
print(f"Fahrenheit: {temp.to_fahrenheit()}°F") # 77.0°F
print(f"Kelvin: {temp.to_kelvin()}K")          # 298.15K
```

8. Methods vs Functions: What's the Difference?

You might be wondering: aren't methods just functions?

Yes, but with an important difference:

Functions are standalone:

```
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
```

Methods belong to a class and have access to the object's state:

```
class Greeter:
    def __init__(self, greeting):
        self.greeting = greeting

    def greet(self, name):
        print(f"{self.greeting}, {name}!")

greeter = Greeter("Howdy")
greeter.greet("Alice") # Howdy, Alice!
```

Key difference: Methods have access to `self` and can use instance data.

Key Takeaways

- ✓ **Methods** are functions inside classes that can access object state
 - ✓ **`self` means "this specific object"** - two ways to understand it:
 - It's a reference to the current object
 - Python automatically passes the object as first argument
 - ✓ **State** is all the instance variable values at any moment
 - ✓ **Methods access state** through `self.variable_name`
 - ✓ **`__str__`** provides human-readable string representation (for `print()`)
 - ✓ **`__repr__`** provides developer-friendly representation (for debugging)
 - ✓ **Special methods** (`__method__`) customize object behavior
 - ✓ **Other languages use `this`** instead of `self` (same concept)
 - ✓ **Methods can return values** just like regular functions
 - ✓ **Many more special methods exist** - we'll learn them in advanced classes
-

Reflection Questions

1. What's the difference between a method and a function?
2. What does `self` represent in a method?
3. Why does every method have `self` as the first parameter?
4. What is an object's "state"?
5. What's the difference between `__str__` and `__repr__`?
6. When is `__str__` automatically called?

7. What does Python do when you call `object.method()`?
 8. How do methods access instance variables?
-

Practice Exercises

1. **Calculator Class:** Create a class that stores two numbers and has methods for add, subtract, multiply, divide
2. **Circle Class:** Store radius, add methods for `area()` and `circumference()`, include `__str__`
3. **Counter Class:** Start at 0, add methods `increment()`, `decrement()`, `reset()`, `value()`
4. **Product Class:** Store name, price, quantity. Add method `total_value()` and proper `__str__`
5. **Player Class:** Store name, score. Add methods `add_points()`, `remove_points()`, include `__repr__`
6. **Library Book:** Store title, author, checked out status. Add `check_out()` and `return_book()` methods
7. **Dice Class:** Store number of sides. Add `roll()` method that returns random number, include `__str__`
8. **Password Class:** Store password. Add methods `check_strength()`, `is_valid()`, include both `__str__` and `__repr__`